Demo runbook: Yanshee

The live demo on the UBTECH Yanshee. This replaces the simulation-only `DEMO_RUNBOOK.md` for the demo day; that file is kept as the record of the original plan and is still the fallback if the robot fails.

Total: about 6 minutes, three short runs on the robot plus one figure.

What the audience should take away

One behaviour cannot suit everyone. The robot learned a different greeting for each kind of person, in simulation, and the same code now drives a real robot. When it uses the wrong person's policy, the person walks away.

Inputs and outputs, in one breath

- **Robot sees:** how far away the person is (4 zones), and its own current greeting strength (low, medium, high). That is 12 situations.
- **Robot does:** step closer, step back, greet more strongly, greet more softly, or hold. 5 actions.
- **Robot is rewarded by:** a touch on its sensor (+1, the person is happy), the person leaving (-2), and a small cost per step (-0.05).
- **Robot learns:** a 12 x 5 table of how good each action is in each situation (tabular Q-learning), one table per person.

The table, scaled down

Yanshee is a 37 cm desktop robot, so the proxemic zones are scaled by 1/3 onto a table. The robot starts 50 cm from the person.

zone	real-world	on the table
intimate	< 0.45 m	< 15 cm
personal	0.45-0.90 m	15-30 cm
social	0.90-1.50 m	30-50 cm
far	> 1.50 m	> 50 cm

Clear at least **80 cm of table** in the robot's line of travel, starting at the person, with nothing near the edges. Never let the robot walk towards an edge.

Playing a user

The users in the study are simulated. On the day, a person plays one, following its rules exactly. Say so openly: the person stands in for the simulated user; the robot is not told who it is.

user	touch the sensor when the robot is…	lean away / leave when closer than
reserved	30-50 cm away **and** greeting quietly	20 cm
neutral	15-30 cm away **and** greeting at medium	12 cm
sociable	under 15 cm **and** greeting loudly	(never)

Touch almost every step while the robot is in your comfortable zone at the right volume; touch only now and then if the distance is right but the volume is not; never touch otherwise. To "leave", take your hand and body out of the sensor's line so it reads nobody there.

Before the room

```
cd nao-adaptive-rl
source .venv/bin/activate
python -m pytest tests -q                          # 24 tests, a few seconds
python -m experiments.yanshee_demo --profile reserved --dry-run    # no robot needed
```

Then, with the robot on the same Wi-Fi as the laptop and charged:

```
python -m experiments.yanshee_check --ip <IP> --only sensors      # readings move?
```

Write the IP and the flags decided in the robot session here:

`IP: _____` `--walk-repeat ___` `mode: sensors / --operator-touch / --operator`

Have `results/personalization_comparison.png` open in a tab.

The 30-second framing

> "A robot that greets everyone the same way will be wrong for most people. Some like you close and loud, some want distance and quiet. We asked whether a robot can learn each person's preference from their reactions alone, and whether that beats the best single behaviour you could ship. It learned in simulation; the same code now runs on this Yanshee."

Run 1: a reserved person (about 1 minute)

```
python -m experiments.yanshee_demo --profile reserved --ip <IP> --steps 10
```

Play **reserved**. The robot takes one step, stops in the social zone, and greets quietly. Touch it as it holds.

> "It stopped at a distance and kept its voice down. Nobody programmed 30 cm; it learned that from touches and walk-aways."

Run 2: a sociable person (about 1.5 minutes)

Place the robot back at the start (the script asks).

```
python -m experiments.yanshee_demo --profile sociable --ip <IP> --steps 12
```

Play **sociable**. It raises its voice, walks right up, and greets loudly.

> "Same algorithm, same robot, a different person, and the opposite behaviour."

Run 3: the wrong policy (about 1 minute)

```
python -m experiments.yanshee_demo --profile sociable --ip <IP> --steps 10
```

Play **reserved** this time. The robot approaches as it would for a sociable person; lean away when it is closer than 20 cm.

> "That is the result of the study in one interaction. The best single policy we found is optimal for one kind of user and worse than doing nothing for the other two. Without personalisation, most users walk away."

The numbers (30 seconds, the saved figure)

Show `results/personalization_comparison.png`:

condition	reserved	neutral	sociable
scripted baseline	-1.26	2.03	-0.75
best single policy	11.81	-0.75	-0.75
one policy per user	**11.83**	**10.31**	**8.34**

Mean reward per interaction, 8 seeds, 6000 episodes.

Questions to expect

"Is it learning right now?" No. Learning one person's policy takes about 2,300 interactions, hours of someone standing at a table. The policies are learned in simulation (about 1 second each, printed at the start of every run) and run on the robot unchanged. That is how most robot learning is deployed.

"How does it know which person it is greeting?" We tell it, as every personalised condition in the study assumes. Recognising the person (Yanshee has face recognition) would be the next step, not part of this project.

"Why did one-policy-for-everyone do so badly?" Different people give opposite feedback in the same situation, so a single policy can only satisfy one of them. The best single policy is the strongest opponent we could build: each user's own policy, tried on everyone, keeping the best.

"Did pre-training on other people help a new person?" No, and we report it. In this task what to do *is* the preference, so there is nothing general to carry over. That is the negative result in `results/adaptation_speed.png`.

"Is any of this staged?" `python -m pytest tests -q -k single_sense` runs the test that guarantees the robot reads the person exactly once per step (an earlier bug read twice and inflated the results).

Limits, say them before being asked

- The users are **simulated**, and on the day a person plays one by its rules. This shows that the learned behaviour transfers to a robot, not how real people would react.
- **Tabular Q-learning** on 12 states. Chosen because it is interpretable and trains in a second; deep RL would be needed for richer sensing.
- **Three user types**, not a population; no human study and no subjective measures.
- The distance zones are **scaled to a tabletop** for a 37 cm robot.

If something fails

what fails	do this
touch sensor	add `--operator-touch` and type `t` when touched
infrared and touch	add `--operator`: type the zone (`i p s f`), `t` for touch, `l` for left
the robot	`--dry-run`: the same runs on the laptop, the fake robot printing its actions
everything	the figures in `results/` and the table above carry the whole story